

Costruttori

Costruttori

Come inizializzare un oggetto quando viene creato.

Il problema da risolvere

Nella pagina precedente creavamo un oggetto e poi riempivamo i campi:

```
Persona persona = new Persona();
persona.nome = "Luca";
persona.eta = 20;
```

Funziona, ma è facile dimenticare un campo.

Un **costruttore** serve a inizializzare l'oggetto nel momento in cui viene creato.

Creare un costruttore

```
public class Persona {
    String nome;
    int eta;

    public Persona(String nome, int eta) {
        this.nome = nome;
        this.eta = eta;
    }
}
```

Il costruttore ha lo stesso nome della classe.

Non ha tipo di ritorno: non scrivi ne `void` ne `int` ne altro.

Usare il costruttore

```
public class Esempio {  
    public static void main(String[] args) {  
        Persona persona = new Persona("Luca", 20);  
  
        System.out.println(persona.nome);  
        System.out.println(persona.eta);  
    }  
}
```

Output:

```
Luca  
20
```

I valori `"Luca"` e `20` vengono passati al costruttore.

A cosa serve this

Nel costruttore abbiamo scritto:

```
this.nome = nome;
```

Qui ci sono due `nome` :

- `this.nome` e il campo dell'oggetto
- `nome` e il parametro del costruttore

`this` significa "questo oggetto".

La riga si legge così: "metti nel campo `nome` di questo oggetto il valore ricevuto nel parametro `nome`".

Costruttore predefinito

Se non scrivi nessun costruttore, Java ne crea uno vuoto per te.

```
public class Persona {  
    String nome;  
    int eta;  
}
```

Puoi fare:

```
Persona persona = new Persona();
```

Ma appena scrivi un costruttore con parametri, Java non crea più automaticamente quello vuoto.

Piu costruttori

Puoi avere più costruttori con parametri diversi.

```
public class Persona {  
    String nome;  
    int eta;  
  
    public Persona() {  
        nome = "Sconosciuto";  
        eta = 0;  
    }  
  
    public Persona(String nome, int eta) {  
        this.nome = nome;  
        this.eta = eta;  
    }  
}
```

Uso:

```
Persona a = new Persona();  
Persona b = new Persona("Sara", 25);
```

Questo è un caso di overloading applicato ai costruttori.

Esempio completo

```
public class Prodotto {  
    String nome;  
    double prezzo;  
  
    public Prodotto(String nome, double prezzo) {  
        this.nome = nome;  
        this.prezzo = prezzo;  
    }  
  
    void mostra() {  
        System.out.println(nome + ": " + prezzo + " euro");  
    }  
}
```

```
public class Negozio {  
    public static void main(String[] args) {  
        Prodotto quaderno = new Prodotto("Quaderno", 2.50);  
        Prodotto penna = new Prodotto("Penna", 1.20);  
  
        quaderno.mostra();  
        penna.mostra();  
    }  
}
```

I costruttori rendono più difficile creare oggetti incompleti.